

CMD (Command Line Execution)

Milenko Burlica

28 July, 2026

Table of Contents

1	Core Engine: CMD (Command Line Execution).....	3
1.1	Globales Feature: condition (Bedingte Ausführung)	3
1.2	Unterstützte Operationen (op)	3
1.2.1	1. Operation: EXEC (Kommando-Ausführung)	3
1.2.2	2. Operation: ASSERT (Validierung)	4
1.2.3	3. Operation: BUFFER (Datenextraktion)	4
1.3	💡 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling	5
1.3.1	2. Das Problem mit dem Doppelpunkt :	5
1.3.2	3. Maskierung (Escaping) von Anführungszeichen	6
1.3.3	4. Mehrzeilige Befehle (Die Block-Notation).....	6

1 Core Engine: CMD (Command Line Execution)

Die `CMD`-Engine dient zur Ausführung, Validierung und Verarbeitung von lokalen Betriebssystem-Befehlen (`cmd.exe` unter Windows) über einfache YAML-Konfigurationen.

Ein zentrales Merkmal ist die **Entkopplung von Ausführung und Validierung**: Ein Befehl wird einmalig ausgeführt (`op: EXEC`) und das Ergebnis unter einem frei wählbaren Namen (`response`) gespeichert. Anschließend können beliebig viele Prüfungen (`op: ASSERT`) oder Datenextraktionen (`op: BUFFER`) auf diesem gespeicherten Ergebnis durchgeführt werden, ohne den Befehl erneut aufzurufen.

1.1 Globales Feature: `condition` (Bedingte Ausführung)

Mit dem optionalen Feld `condition` kann gesteuert werden, ob ein Testschritt überhaupt ausgeführt werden soll. Wenn die Bedingung nicht zutrifft (`false`), wird der Schritt einfach übersprungen (`SKIPPED`) und der Test läuft ohne Fehler weiter.

- **Dynamische Werte:** In der Bedingung können alle Platzhalter (wie `{B[...]}` für Variablen oder `{E[...]}` für Umgebungsvariablen) verwendet werden.
- **Wichtig für Strings:** Da Textwerte direkt eingesetzt werden, müssen Text-Variablen in einfache Anführungszeichen `'...'` gesetzt werden.

YAML

```
# Wird nur auf Windows-Systemen ausgeführt
- type: CMD
  condition: '{E[OS_PLATFORM]}' == 'windows'
  op: EXEC
  command: "Scripts\win_backup.bat"
  response: backup_out
```

1.2 Unterstützte Operationen (`op`)

1.2.1 1. Operation: EXEC (Kommando-Ausführung)

Führt ein System- oder CLI-Kommando aus und speichert den gesamten Text-Output sowie den Exit-Code unter dem in `response` definierten Namen ab.

- **Automatischer Arbeitsordner (WORKDIR):** Wenn das Kommando mit einem Laufwerksbuchstaben beginnt (z. B. `C:\Scripts\...`), erkennt das System dies automatisch und führt den Befehl direkt in diesem Verzeichnis aus. Das verhindert Fehler mit relativen Pfaden.

- **Variablen-Auflösung:** Im Feld `command` können alle Dynamic-Tags des Frameworks genutzt werden (`{B[...]}` für TestCase-Variablen, `{R[...]}` für CALL-Parameter, `{T[...]}` für Tabellen-Werte, `{E[...]}` für Umgebungsvariablen).

YAML

```
- type: CMD
  op: EXEC
  command: "java -version"
  response: java_out
```

1.2.2 2. Operation: ASSERT (Validierung)

Prüft den Inhalt oder den Status (Exit-Code) eines zuvor gespeicherten Befehls-Outputs. Schlägt eine Prüfung fehl, bricht der gesamte Test sofort ab (**Fail-Fast**).

Aktion (action)	Beschreibung	Verwendung im YAML
EXITCODE	Prüft den Rückgabewert des Prozesses (Standard-Erfolg ist <code>0</code>).	<code>expected: 0</code>
CONTAINS	Prüft, ob ein Text im Output existiert.	<code>value: "Suchtext"</code>
NOT_CONTAINS	Prüft, ob ein Text nicht im Output existiert.	<code>value:</code> <code>"Fehlermeldung"</code>
COUNT	Zählt, wie oft ein Text im Output vorkommt.	<code>value: "Text"</code> <code>expected: 1</code>

YAML

```
# Beispiel: Inhaltsprüfung auf der zuvor gespeicherten "java_out" Response
- type: CMD
  op: ASSERT
  response: java_out
  action: CONTAINS
  value: "64-Bit Server VM"
```

1.2.3 3. Operation: BUFFER (Datenextraktion)

Liest gezielt Daten aus einem gespeicherten Output und speichert sie in eine neue Variable (`name`), damit nachfolgende Schritte (auch in SQL- oder REST-Engines) darauf zugreifen können.

- **TEXT** : Speichert den kompletten, getrimmten Text-Output.
- **FILTER** : Speichert nur Zeilen, die den Suchbegriff (`value`) enthalten.
- **LINE** : Extrahiert eine exakte Zeile über einen Index (beginnend bei `0` für die erste Zeile).
- **LAST_LINE** : Extrahiert eine Zeile relativ vom Ende (`value: 0` ist die letzte Zeile, `value: 1` die vorletzte).

YAML

```
# Extrahiert die allererste Zeile des Java-Outputs in die Variable
"java_first_line"
- type: CMD
  op: BUFFER
  response: java_out
  action: LINE
  value: 0
  name: java_first_line
```

1.3 💡 Wichtige YAML-Syntaxregeln & Sonderzeichen-Handling

Das Schreiben von Systembefehlen in YAML kann tückisch sein, da Zeichen wie `{`, `}`, `*`, `:` oder `"` für YAML eine spezielle Bedeutung haben. Damit die Tests fehlerfrei laufen, sollten die folgenden Regeln beachtet werden:

- 1. Warum müssen geschweifte Klammern `{ }` in Anführungszeichen?**

In YAML leiten geschweifte Klammern am Zeilenanfang ein Inline-Objekt (eine Map) ein. Wenn du einen Platzhalter wie `{B[my_var]}` als Wert einträgst, wirft der YAML-Parser einen Syntaxfehler.

- **Falsch:** `command: {B[my_script]}` (Parser stürzt ab)
- **Richtig:** `command: "{B[my_script]}"` (Als String geschützt)

1.3.1 2. Das Problem mit dem Doppelpunkt `:`

Der Doppelpunkt ist das Trennzeichen zwischen Schlüssel und Wert in YAML. Wenn dein Befehl einen Doppelpunkt enthält (z. B. bei Windows-Pfaden oder URLs), *muss* der gesamte Wert in Anführungszeichen gesetzt werden.

- **Falsch:** `command: C:\Tools\run.bat`
- **Richtig:** `command: "C:\Tools\run.bat"`

1.3.2 3. Maskierung (Escaping) von Anführungszeichen

Wenn dein CLI-Befehl selbst doppelte Anführungszeichen benötigt (z. B. um einen Parameter mit Leerzeichen an ein Skript zu übergeben), hast du zwei saubere Optionen:

- **Option A (Einfache äußere Anführungszeichen):** Uviješ cijeli string u jednostruke navodnike (' '), tada unutar stringa možeš slobodno koristiti duple navodnike.

YAML

```
command: 'echo "Das ist ein Text mit Leerzeichen"'
```

* **Option B (Backslash-Escaping):** Ako koristiš duple navodnike spolja, unutrašnje duple navodnike moraš eskejpovati pomoću `\"`.

``yaml

```
command: "echo \"Das ist ein Text mit Leerzeichen\""
```

1.3.3 4. Mehrzeilige Befehle (Die Block-Notation |)

Wenn ein Befehl sehr lang ist oder du logisch mehrere Befehle nacheinander ausführen willst, nutze das Pipe-Zeichen (|). Dadurch liest YAML alles darunter als einen einzigen Textblock und du musst dir **keine Sorgen um Anführungszeichen oder Sonderzeichen** machen.

YAML

```
- type: CMD
  op: EXEC
  command: |
    cd C:\Services\App
    run_install.bat --mode=silent --dir="C:\Program Files\App"
  response: install_out
```